

Complete answers + follow-up Q&A — Walmart (4 YOE) Data Engineering interview

Below are well-structured, interview-ready answers for every question in the post, plus realistic follow-up questions you may be asked and strong concise responses for each. Use these as templates: speak clearly, draw diagrams when asked, and adapt details to your real experience.

Round 1 — ETL Pipelines, PySpark & Airflow

Q1 — PySpark Optimization: How would you handle a severe data skew in a join operation? (Think salting, repartitioning strategies).

Short answer (one-liner)

Detect the skew, handle hot keys (salting / broadcast / map-side join / partial aggregation), and/or redesign the join using partitioning and adaptive execution to balance data across tasks.

Detailed approach (steps you can explain)

1. Confirm it's skew

Examine stage/task durations in Spark UI. One or few tasks taking much longer = skew.

Check hot key frequency: `run df.groupBy(key).count().orderBy(desc("count")).show()`.

2. If small dimension / small side → Broadcast join

Use `broadcast(small_df)` so join runs map-side and avoids shuffle. Explain trade-off: small side must fit driver/executor memory.

3. Salting (for heavy keys)

Add a salt column to both sides for the problematic key:

$n = 10$ salts → `salt = floor(rand()*n)` on big table, and replicate small table keys n times with salt $0..n-1$.

Join on (key, salt) so heavy-key rows are split across n partitions.

Trade-offs: increases data size (especially for small table replication) and requires deterministic salt.

4. Skew-aware partitioning / custom partitioner

Use a custom partition function that spreads popular keys across partitions (e.g., hash(key + salt)).

In Spark, use repartition with a composite key or use mapPartitions for manual handling.

5. Partial aggregation before join

Pre-aggregate the big table by key to reduce rows joined, then join to small dimension.

6. Adaptive Query Execution (AQE)

If using Spark 3+, enable AQE to coalesce shuffle partitions and optimize skew-affected stages.

7. Fallback: change algorithm / two-phase join

For extreme skew, isolate hot-key processing separately (process hot keys with specialized logic, and normal keys with regular join).

Example (pseudo-code: salting)

```
n = 10
big = big.withColumn("salt", (rand()*n).cast("int"))
small = small.withColumn("salt", explode(array([lit(i) for i in range(n)])))
joined = big.join(small, ["key", "salt"])
```

Follow-up Qs & answers

Q: How do you pick n for salting?

A: Start small (4–8). Monitor task times and memory. Increase until skew is balanced. Consider executor memory and shuffle cost.

Q: When not to broadcast?

A: If the small table > executor memory or when the small table is changing frequently and broadcast causes stale data across tasks.

Q: How to detect skew without Spark UI?

A: Use `groupBy(key).count()`, sample task logs, or record per-partition counts (`rdd.mapPartitionsWithIndex`) to find imbalance.

Q2 — Memory Management: Explain how you would debug a Spark job failing with OutOfMemory (OOM) errors.

Short answer

Use Spark UI and executor logs to locate OOM stage, then reduce memory pressure via persistence strategy, increase shuffle partitions, reduce data skew, use broadcast wisely, optimize joins/serializers, or increase executors/memory.

Step-by-step debugging & fixes

1. Locate OOM

Check driver/executor logs and Spark UI for the failing stage and task logs. Identify whether OOM is on driver or executor and which operation (shuffle, join, collect, cache).

2. Common causes

Large `collect()` or `toPandas()`, large broadcast, caching too many partitions, skew, too-large shuffle partitions, high parallelism with limited executors.

3. Immediate fixes

Avoid `.collect()` or limit it. Use `.take(n)` for small samples.

Use `persist(StorageLevel.MEMORY_AND_DISK)` instead of `MEMORY_ONLY`.

Increase `spark.sql.shuffle.partitions` (e.g., from 200 to 1000) to reduce per-task shuffle size.

Increase executor memory/cores if cluster allows.

4. Optimize data structures & serialization

Use Kryo serializer and register classes.

Avoid nested complex types and wide schemas when not necessary.

5. Algorithmic fixes

Use broadcast join only if broadcast side is small.

Pre-aggregate or filter early to reduce dataset size.

Use map-side combine for `reduceByKey` vs `groupByKey`.

6. Resource tuning

Tune `spark.executor.memory`, `spark.executor.cores`, `spark.driver.memory`, and off-heap memory with `spark.memory.offHeap.enabled`.

7. Long-term

Add unit tests on sampled datasets, set up job-level memory alerting, create a performance baseline in CI.

Example debugging sequence

Spark UI shows OOM in shuffle write on executor X → increase shuffle partitions; check if a single key is heavy → if yes, handle skew.

Follow-ups

Q: Why MEMORY_AND_DISK?

A: It spills partitions to disk when memory is insufficient, preventing OOM at cost of I/O.

Q: When to use Kryo?

A: When default Java serializer is slow or large; Kryo is faster and more compact but requires class registration for best performance.

Q3 — Data Modeling in Code: Design SCD Type 2 pipeline using PySpark.

Requirements to state up-front

Source incremental data (CDC or daily delta).

Target dimension table keeps history with start_date, end_date (or is_current + effective_from), surrogate_key.

Need idempotency, concurrency handling, and efficient updates (merge).

High-level pipeline (steps)

1. Extract: Read source incremental file (new/changed rows) with business key bk and attributes.

2. Normalize / Hash: Compute a digest/hash of attribute values to detect changes: digest = sha2(concat_ws('|', attr1, attr2,...), 256).

3. Read current dimension: Read dim current data from the warehouse (SCD table).

4. Join: Left-join incoming to dim on business key.

5. Classify rows:

No match → insert (new row) with `is_current = true`, `effective_from = today`, `effective_to = null`.

Match & digest same → no action (no change).

Match & digest differs → update current row `is_current=false`, set `effective_to = yesterday`, and insert new row with `is_current=true`.

6. Write: Use atomic MERGE (Delta Lake/SQL MERGE) or transactional update. If using plain Parquet, implement staging and swap tables with transactional metadata.

Example pseudo-code (Delta Lake MERGE)

```
MERGE INTO dim d
USING updates u
ON d.business_key = u.business_key AND d.is_current = true
WHEN MATCHED AND d.digest <> u.digest THEN
  UPDATE SET is_current = false, effective_to = u.effective_from - interval 1 day
WHEN NOT MATCHED THEN
  INSERT (surrogate_key, business_key, ..., digest, is_current, effective_from, effective_to)
  VALUES (...)
```

If MERGE not available

Create `to_update` and `to_insert` sets, write inserts, and mark old rows inactive in a transaction-like manner.

Operational concerns

Idempotency: Use `digest` and `effective_from` to avoid double applying the same change. Store `source_file_id` or `cdc_offset` for replay safety.

Backfill / corrections: Write procedures to rebuild dimension if needed.

Performance: Partition dimension on hashed business key or bucket; keep current rows separate for faster joins.

Follow-ups

Q: How to handle late-arriving data?

A: Accept late events via a reprocessing window. Use effective_from from source if available, or append and correct effective_to ranges during reprocessing.

Q: How to scale MERGE on very large dims?

A: Use partition pruning (partition on business key hash), use Delta/Z-ordering, or split updates into hot-key and cold-key processing.

Round 2 — SQL + Python Data Handling

Q4 — Advanced SQL: Write a query to sessionize user activity data with a 30-minute timeout.

Problem restatement

Given event-level table with user_id, event_time (timestamp), event_type, create sessions where any gap > 30 minutes ends the session. Output: user_id, session_id, session_start, session_end, events_count.

Solution using window functions (ANSI SQL / BigQuery / Postgres style)

```
WITH events AS (  
  SELECT  
    user_id,  
    event_time,  
    event_type,  
    LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) AS prev_time  
  FROM raw_events  
)  
flag AS (  
  SELECT  
    user_id,  
    event_time,  
    CASE  
      WHEN prev_time IS NULL THEN 1  
      WHEN EXTRACT(EPOCH FROM (event_time - prev_time)) > 1800 THEN 1  
      ELSE 0
```

```

        END AS is_new_session
    FROM events
),
sessionized AS (
    SELECT
        user_id,
        event_time,
        SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY event_time ROWS
UNBOUNDED PRECEDING) AS session_num
    FROM flag
)
SELECT
    user_id,
    CONCAT(user_id, '_', session_num) AS session_id,
    MIN(event_time) AS session_start,
    MAX(event_time) AS session_end,
    COUNT(*) AS events_count
FROM sessionized
GROUP BY user_id, session_num
ORDER BY user_id, session_start;

```

Notes:

Use `EXTRACT(EPOCH FROM ...) > 1800` for 30 minutes. In BigQuery use `TIMESTAMP_DIFF(event_time, prev_time, SECOND) > 1800`.

For heavy data, partition and cluster on `user_id`.

Follow-ups

Q: How to include session-level metrics like unique pages?

A: Use `COUNT(DISTINCT page)` inside the final `SELECT` grouped by session.

Q: How to handle out-of-order events?

A: Buffer events by an ingestion watermark (allow lateness) or use event-time windows in streaming engines with watermarking.

Q5 — Python Concurrency: Script that uses multiprocessing/multithreading to download and validate many files from S3.

Key design points

Use concurrency for I/O-bound downloads (threads) or CPU-bound validations (processes).

Use a producer-consumer pattern and robust retries with exponential backoff.

Ensure rate limits and S3 best practices (s3 clients are thread-safe for boto3 but you may create session per thread).

Provide metrics and idempotency (e.g., store downloaded file path + checksum).

Example hybrid approach (download with threads, validate with processes)

```
import boto3, botocore
import concurrent.futures
import hashlib
from multiprocessing import Pool
from pathlib import Path
import time

s3 = boto3.client('s3')

def download_one(s3_key):
    local = Path('/tmp') / Path(s3_key).name
    for attempt in range(3):
        try:
            s3.download_file('bucket-name', s3_key, str(local))
            return str(local)
        except botocore.exceptions.ClientError as e:
            time.sleep(2**attempt)
    raise RuntimeError(f"Failed {s3_key}")

def validate_file(local_path):
    # CPU-bound example: compute sha256
    h = hashlib.sha256()
    with open(local_path, 'rb') as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return local_path, h.hexdigest()

def process_keys(s3_keys):
    # Download concurrently (I/O-bound) with ThreadPoolExecutor
```

```
with concurrent.futures.ThreadPoolExecutor(max_workers=20) as download_pool:
    futures = {download_pool.submit(download_one, k): k for k in s3_keys}
    local_files = []
    for fut in concurrent.futures.as_completed(futures):
        local_files.append(fut.result())

# Validate concurrently (CPU-bound) with multiprocessing Pool
with Pool(processes=4) as pool:
    results = pool.map(validate_file, local_files)

return results
```

Operational best practices

Use chunked streaming (S3 TransferConfig) for large files.

Save metadata (file size, checksum, timestamp) to a central store for idempotency.

Rate-limit and backoff to avoid throttling.

Monitor failures and create a dead-letter queue.

Follow-ups

Q: Threading vs Multiprocessing?

A: Threading for I/O-bound (network), multiprocessing for CPU-bound (hashing, parsing).

Combining both gives best throughput.

Q: How to handle partial downloads or corrupted files?

A: Check content length and checksum; if mismatch, retry and optionally move to quarantine for manual inspection.

Q6 — Data Quality Checks: Implement a robust data quality check framework within Python/PySpark before writing data.

Requirements & goals

Validate schema, nulls, ranges, uniqueness, referential integrity, row counts, and business rules.

Produce human-readable reports and machine-readable metrics, and optionally block pipelines on critical failures.

Framework design (components)

1. Checks library – set of reusable check functions (e.g., `check_not_null(col)`, `check_unique(col)`, `check_range(col,min,max)`).
2. Rules config – JSON/YAML that declares checks per table/column and thresholds (critical, warning).
3. Runner – code that executes checks on DataFrame and collects results.
4. Reporting – store results in a DQ table, send alerts for critical errors, produce detailed CSV/HTML report.
5. Action – policy to fail/alert/skip/write-to-quarantine.

Example checks (PySpark)

```
def check_not_null(df, col):
    total = df.count()
    nulls = df.filter(df[col].isNull()).count()
    return {"check": "not_null", "col": col, "total": total, "failures": nulls, "pass": nulls==0}
```

```
def check_uniqueness(df, cols):
    total = df.count()
    dup = df.groupBy(cols).count().filter("count > 1").count()
    return {"check": "unique", "cols": cols, "total": total, "duplicates": dup, "pass": dup==0}
```

```
# Runner
results = []
results.append(check_not_null(df, 'user_id'))
results.append(check_uniqueness(df, ['user_id', 'event_time']))
# Write results to DQ table and decide action
```

Using existing tools (mention for interview)

Great Expectations, Deequ (AWS), or custom rules stored in DB. (If asked, explain limitations and integration approach.)

Follow-ups

Q: How do you set thresholds?

A: Use historical baselines and business SLAs. For example, if historically nulls <0.1% treat 1% as warning, >5% as failure.

Q: How to test your DQ framework?

A: Unit tests on synthetic data, integration tests in CI with sample datasets, and run on a staging dataset that mimics production.

Round 3 — System Design + Data Quality

Q7 — Batch Processing Architecture: Design a fault-tolerant system to ingest daily sales data from thousands of retail stores into a central DWH.

Goals & constraints to state

High ingest throughput, fault-tolerant, idempotent, eventual consistency acceptable (daily), low operational cost, monitorability, and easy reprocessing.

Proposed architecture (components & data flow)

1. Edge collection (store level)

Each store writes daily sales files to an object store (S3 / ADLS / GCS) using secure upload — files named with store_id/date/uuid.

Alternatively use lightweight agents that push to central ingestion API.

2. Landing zone

Raw files land in S3 raw/sales/YYYY/MM/DD/store_id/filename. Use object metadata: file_checksum, file_size, source_ts, file_id.

3. Ingestion orchestration

Airflow job scheduled daily: lists new files, validates checksums, and moves to staging. Use idempotent catalog table storing processed file IDs.

4. Preprocessing & validation

Run PySpark jobs to validate schema, checksums, and business rules (DQ framework). Failed files go to quarantine and trigger alerts.

5. Transformation

Clean, normalize, enrich (store metadata lookup), aggregate if required, partition by date/store.

6. Storage

Write transformed data to raw zone (parquet) and then to warehouse (Redshift/BigQuery/Snowflake) or Delta Lake for downstream analytics. Use partitioning on date and distribution key store_id.

7. Load to DWH

For DWH: bulk load from object store using COPY/LOAD (fast). For Lakehouse: write Delta tables and use MERGE for slowly changing data.

8. Idempotency & exactly-once

Track ingested_files table; only process files not in table. Use file-level checksums and atomic insert of ingestion metadata.

9. Failure handling

Retry transient failures, move bad files to quarantine, and store logs. Use backfill pipelines to replay from raw or from source.

10. Monitoring & alerting

Metrics: file counts, bytes processed, per-store failure rate. Integrate with PagerDuty/Slack. Keep audit logs.

Fault tolerance specifics

Retries & exponential backoff for network or API calls.

Checkpointing for long Spark jobs (if streaming-like needs).

Atomic writes & transaction logs (Delta/ACID tables) to avoid partial writes.

Parallelism & partitioning tuned to throughput.

Follow-ups

Q: How to ensure low-latency if business needs near-real-time?

A: Add a streaming ingestion path (Kafka/Kinesis) for near-real-time delta; maintain both streaming and batch pipelines feeding the lakehouse with merge logic.

Q: How to handle store-level schema drift?

A: Enforce schema validation with compatibility rules; if optional fields appear, store as JSON column or evolve table schema with controlled migrations.

Q8 — Lakehouse vs Data Warehouse: Architectural differences & when to choose each.

Core differences (concise)

Data Warehouse (Redshift/Snowflake/BigQuery)

Schema-on-write, optimized for structured BI queries, strong performance for analytic SQL, usually managed columnar databases.

Good for stable schemas, strict governance, complex SQL analytics.

Lakehouse (Databricks/Delta Lake)

Combines low-cost object storage with ACID transactions (Delta), supports both batch & streaming, schema evolution, and supports data science workloads with large semi-structured data.

Good for advanced ML, data science, ELT patterns, unstructured/semi-structured data.

When to choose

Choose Warehouse when**: primarily BI/reporting, strict concurrency/managed service, simple ELT patterns, and budget allows.

Choose Lakehouse when**: you need to handle streaming + batch, machine learning workflows, large volumes of semi/unstructured data, or you want unified storage and compute separation.

Follow-ups

Q: Cost differences?

A: Warehouses can be more expensive per TB for storage & query cost; lakehouses leverage cheaper object storage and scale compute separately. But total cost depends on usage patterns and concurrency.

Q: How to migrate from a DWH to Lakehouse?

A: Export data to object store, create Delta tables, test queries, gradually move workloads and validate SLAs.

Q9 — Use Case: Design a data pipeline for a machine learning model that needs daily retraining with fresh data.

Requirements to state

Daily retrain, reproducible training runs, feature freshness, model versioning, rollback, resource efficiency.

Pipeline components

1. Data ingestion & feature generation

ETL job generates daily features and stores them in a feature store or parquet tables partitioned by date.

2. Feature store

Serve features for both training (historical) and online inference (fresh values). Ensure consistency between training and serving features.

3. Training orchestration

Orchestrate retrain job in Airflow (or similar) daily: fetch latest training data, train model, evaluate via test set and validations.

4. Model validation & evaluation

Evaluate against holdout sets and business metrics (e.g., precision, recall, expected revenue uplift). Include data drift detection.

5. Model registry & versioning

If model passes checks, register artifact in model registry (MLflow) with metadata (data snapshot id, metrics, code hash).

6. Deployment

Canary deploy to a fraction of traffic or shadow mode. Monitor online metrics and rollback if degradation.

7. Monitoring & alerting

Data drift, model performance, inference latency, and biases monitored. Revert to previous model if needed.

8. Reproducibility

Store training data snapshot id (e.g., date=YYYY-MM-DD), hyperparameters, random seeds, and code version.

Follow-ups

Q: How to avoid label leakage?

A: Use temporal boundaries, ensure features are generated only from data available at prediction time, and use backtesting with realistic time splits.

Q: How to manage compute cost?

A: Use spot instances, incremental training if possible, and schedule retraining in off-peak hours with resource autoscaling.

Round 4 — Leadership & Project Round

Q10 — Technical Decision Making: Describe a time you had to pick a new tool/framework. Walk through process and trade-offs.

Answer structure (STAR + trade-off summary)

1. Situation: Briefly set context (project constraints: team size, timeline, scale).
2. Task: What decision was needed and why.
3. Action: How you evaluated (POC, cost, performance, learning curve, community, security, vendor lock-in).
4. Result: Outcome, adoption path, and retrospective.

Example answer (template you can adapt)

Situation: Our ETL pipeline was running slowly and costs were growing on legacy VMs.

Task: Choose between moving to managed Spark (Databricks) or refactoring to Presto-based query engines.

Action: Ran a 2-week POC for both. Measured TCO, job latency, team ramp-up, integration with S3 and IAM, data governance. Presented findings with benchmarks and risk analysis to stakeholders. Chosen Databricks for faster development, better Spark optimizations, and built-in Delta Lake. Planned migration in 3 phases with a rollback plan.

Result: 40% decrease in average job runtime and 25% reduction in ops time. We documented playbooks and ran training sessions.

Follow-ups

Q: What trade-offs did you accept?

A: Higher managed service cost vs faster time-to-market and lower ops burden. Also vendor dependency was recognized and mitigated by using open formats (Parquet/Delta).

Q: How did you measure success?

A: Baseline metrics (job runtime, error rate), cost analysis, and team productivity improvements post-migration.

Q11 — Project Management: How do you estimate timelines and handle potential delays in a complex data engineering project?

Estimation approach

1. Breakdown: Decompose into milestones and tasks (ingestion, schema design, ETL dev, QA, infra, monitoring).
2. Expert input: Get estimates from engineers for each task (three-point estimate: optimistic / likely / pessimistic).
3. Dependencies: Map external dependencies (3rd party APIs, infra approvals) and add buffer.
4. Sprint planning: Use iterative approach — deliver MVP quickly and then iterate.
5. Risk register: Identify risks and mitigation strategies.

Handling delays

Early detection: Daily stand-ups, CI pipelines, and intermediate demos.

Mitigation: Re-prioritize scope (defer non-critical features), add parallel workstreams, or negotiate deadline.

Communication: Proactively inform stakeholders with impact and remediation plan.

Postmortem: After delivery, run blameless retrospective and update process.

Follow-ups

Q: How much buffer do you add?

A: Depends on uncertainty. For greenfield ~30% buffer; for stable tasks ~10–15%. Always call out assumptions.

Q: How to handle cross-team dependencies?

A: Early alignment meetings, clear SLAs for handoffs, run integration tests in staging, and add acceptance gates.

Q12 — Learning from Failure: Tell me about a time a project had a major setback and what you learned.

Recommended structure & sample story

Situation: Briefly state the project and the setback (e.g., large data corruption after a failed job).

Action: What you did immediately: rolled back to previous snapshot, quarantined bad data, and notified stakeholders.

Root cause & fix: Root cause was lack of atomic write patterns and missing DQ checks. Implemented transaction-safe writes (Delta/ACID), added pre-write DQ checks, and file-level immutability.

Outcome: Restored trust, implemented monitoring, and updated runbooks.

Lessons: Emphasize systems thinking, automated testing, and the importance of observability.

Follow-ups

Q: How did you prevent recurrence?

A: Implemented staging + validation, automated pre-commit checks, and introduced canary deployments for critical jobs.

Q: How did you communicate with stakeholders?

A: Sent immediate incident report with impact and mitigation; provided daily updates until resolution and a final postmortem.

Quick checklist — interview prep tips

Bring metrics and concrete numbers from your projects (data size, runtimes, improvements).

Be ready to whiteboard partitioning, data flow, and MERGE logic.

Practice one or two live coding tasks (SQL sessionize and a short PySpark snippet).

Prepare 1–2 leadership stories (technical decision, failure + lesson). Use STAR.

For system design, always state assumptions (QPS, data size, SLA) before proposing architecture.